PROJECT

CSE0002

OPEN SOURCE SOFTWARE (LINUX ADMINISTRATION)

SLOT-D11

SUBMITTED BY:

Name : Kreeti Kumari

Reg no. : 24BAI10642

SUBMITTED TO:

Dr. Gunjan Ansari

(Programme Chair Gaming Technology)

School Of Computing Science & Engineering (SCOPE)

VIT Bhopal University

Date of Submission:

28/03/2026

TABLE OF CONTENT

1. **Introduction**

2. **Part-A - Origin And Philosophy**

   A1. The Problem This Software Was Created to Solve

   A2. The License — What It Actually Says

   A3. The Ethics of Open Source

3. **Part B — Linux Footprint**

4. **Part C — The FOSS Ecosystem**

5. **PART D - OPEN SOURCE VS PROPRIETARY**

6. **Project Timeline**

7. **REFERENCES**

8. **CONCLUSION**

9. **SHELL SCRIPTS**

   9.1 Script 1

   9.2 Script 2

   9.3 Script 3

   9.4 Script 4

   9.5 Script 5

INTRODUCTION

Linux and Free, Open Source Software (FOSS) are part of modern computing. Open-source software provides the opportunity for developers to access, modify, and freely share source code which helps to encourage innovation and teamwork among developers.

Some of the most common software that represents FOSS includes Python, a high-level programming language famous for code clarity and readability. Web development, data science, AI and Automation can use Python.

Python is open-source which means its source code is publicly available and can be modified by anybody. Wide and global supportive community, mainly used for scripting and development in Linux systems.

This project aims to provide information relating major concepts of Linux and FOSS (Free & Open Source Software) in open-source software on Python! Including the Hands-On implementation using shell script tasks on Linux.

PART-A - ORIGIN AND PHILOSOPHY

A1. The Problem This Software Was Created to Solve

Prior to the release of open-source software, proprietary systems largely ruled the computing industry. These systems didn't allow users to see or modify the source code and theoretically made it so that people either couldn't understand how an application worked, or they couldn't customize it at all. Therefore, depending on the software companies for any updates, bugs fixes, or feature additions were heavily dependent. This dependency had the dual impact of increasing costs and stagnating innovation, as users could not be involved in polishing the software.

Programming languages like C and C++ ruled the field, but they needed a deep knowledge of complex syntax and memory allocation. This made them less beginner-friendly and it negatively affected overall productivity. It needed an easy language, easy to learn and powerful programming language.

Python is an open-source programming language that was developed to solve these problems, emphasizing simplicity and readability. It provided developers with a very concise and easy-to-read syntax to avoid complexity from the code. Meanwhile, Linux offered a free and open operating system that allowed complete control over the computing environment. Linux and Python solved a lot of pain points in system-level and programming-level computing, all under the themes of accessibility, flexibility, and efficiency.

Meanwhile, Linux offered a free operating system that users could run whatever software they pleased on.

Linux and Python provided solutions to system-level and programming-level problems encapsulating openness, flexibility, and efficiency.

A2. The License — What It Actually Says

Licensing is a fundamental aspect of open-source software, as it defines how software can be used, modified, and distributed. The Linux operating system is released under the **GNU General Public License**, which ensures that the software remains free and open for all users. This license allows users to access the source code, make modifications, and distribute the software, provided that any modified version is also shared under the same license.

Python, on the other hand, is distributed under the **Python Software Foundation License**, which is a more permissive open-source license. It allows users to freely use, modify, and distribute Python, even for commercial purposes, without strict restrictions.

The concept of **copyleft**, which is a key feature of the GPL, ensures that software remains open and prevents it from being converted into proprietary software. These licensing models play a crucial role in maintaining the openness and fairness of software development. They encourage collaboration and ensure that the benefits of software are shared among all users.

A3. The Ethics of Open Source

The open-source movement is not only based on technical principles but also on strong ethical values. It promotes the idea that knowledge should be freely available and that collaboration leads to better results. Open-source software encourages developers to share their work, learn from others, and contribute to a global community.

The core ethical principles of open source include freedom, transparency, collaboration, and accessibility. Freedom ensures that users have control over the software they use. Transparency allows users to examine the source code, which increases trust and security. Collaboration enables developers from different parts of the world to work together, leading to rapid innovation. Accessibility ensures that software is available to everyone, regardless of their financial background.

Organizations such as the Free Software Foundation and the Open Source Initiative play a significant role in promoting these ethical values. Python is a perfect example of these principles in action, as it is developed and maintained by a global community that continuously works to improve it.

PART B — LINUX FOOTPRINT

Linux has established itself as one of the most powerful and widely used operating systems in the modern technological world. Since its development, it has grown from a simple open-source project into a global platform that powers a vast range of devices and systems. The **Linux kernel**, developed by Linus Torvalds, forms the core of this ecosystem and provides a stable and secure foundation for various operating system distributions. Its open-source nature allows developers to modify and customize it according to their needs, which has contributed significantly to its widespread adoption.

One of the most prominent areas where Linux has made a strong impact is in web servers and internet infrastructure. A majority of websites across the world are hosted on Linux-based servers due to their reliability, performance, and security. Linux systems are capable of handling large amounts of traffic efficiently, making them suitable for enterprise-level applications. This has made Linux the backbone of the internet, supporting services such as websites, online applications, and databases.

In addition to web servers, Linux plays a crucial role in cloud computing environments. Major cloud service providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud rely heavily on Linux-based systems to deliver scalable and efficient services. The flexibility of Linux allows organizations to deploy customized solutions and manage resources effectively. Its compatibility with various programming languages, including **Python**, further enhances its usability in cloud-based applications and services.

Linux is also widely used in mobile technology. The Android operating system, which powers the majority of smartphones worldwide, is based on the Linux kernel. This demonstrates the adaptability of Linux in different environments, ranging from powerful servers to small handheld devices. Its efficient resource management and open architecture make it suitable for mobile platforms where performance and customization are essential.

Linux is also widely used in embedded systems and Internet of Things (IoT) devices. Devices such as smart TVs, routers, home automation systems, and industrial machines often use Linux as their operating system. Its lightweight nature and flexibility allow it to be adapted for specific hardware requirements, making it ideal for embedded applications.

Popular Linux distributions such as **Ubuntu**, **Debian**, and **Fedora** provide user-friendly interfaces and tools that cater to different types of users. Ubuntu is known for its simplicity and is widely used by beginners, while Debian is valued for its stability, and Fedora is preferred for its advanced features.

An important aspect of the Linux footprint is the role of Python within the Linux environment. Python is extensively used for automation, system administration, and application development in Linux systems. Many administrative tools and scripts are written in Python due to its simplicity and efficiency. This reduces manual effort and improves productivity.

Furthermore, Linux contributes significantly to reducing the cost of software usage. Since it is open-source, users do not need to pay licensing fees, which makes it an economical choice for individuals and organizations. Its strong security features and transparency make it a reliable and trusted operating system.

In conclusion, the footprint of Linux extends across multiple domains, including servers, cloud computing, mobile devices, and embedded systems. Its flexibility, reliability, and open-source nature have made it a fundamental component of modern technology. The integration of Python further enhances its capabilities, making Linux a powerful platform for development and automation.

The following screenshots confirm Python installation and configuration on the Linux system:

```
kreeti@LAPTOP-N52FFLNN:~$ cd /mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$ python3 --version
Python 3.12.3
```

```
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$ which python3
/usr/bin/python3
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$ command -v python3
/usr/bin/python3
```

```
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$ python3 -c "print('Python is working')"
Python is working
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$
```

PART C — THE FOSS ECOSYSTEM

The Free and Open Source Software (FOSS) ecosystem represents a vast and interconnected network of developers, organizations, tools, and platforms that collectively contribute to the creation and improvement of open-source software. Unlike traditional software development models, which are controlled by a single organization, the FOSS ecosystem is decentralized and community-driven. This allows developers from different parts of the world to collaborate, share ideas, and contribute code, leading to continuous improvement and innovation.

One of the most important components of the FOSS ecosystem is version control systems, particularly Git. Version control systems help developers track changes in their code, manage multiple versions of a project, and collaborate efficiently. Platforms such as GitHub and GitLab provide a collaborative environment where developers can host projects, contribute to open-source initiatives, and work together in teams. These platforms have become central hubs for open-source development.

Another key aspect of the FOSS ecosystem is the availability of programming languages and tools that support development. Among these, **Python** plays a significant role due to its simplicity, readability, and versatility. Python is widely used in areas such as web development, data science, machine learning, and automation. Its large number of libraries, including NumPy, Pandas, and TensorFlow, makes it highly efficient for solving complex problems.

The Python ecosystem is supported by a strong global community that actively contributes to its growth. Developers contribute by creating libraries, fixing bugs, improving documentation, and sharing tutorials. This collaborative approach ensures that Python continues to evolve and remain relevant in modern technology. The availability of free learning resources also makes it easier for beginners to learn and participate in open-source development.

The FOSS ecosystem also promotes innovation by reducing barriers to entry. Since most tools and software are freely available, individuals and organizations can experiment, learn, and develop solutions without significant financial investment. This has led to the growth of startups and new technologies that rely on open-source platforms.

Furthermore, the FOSS ecosystem has a strong impact on education and learning. Students and developers can access open-source tools and resources to gain practical knowledge and skills. **Python**, in particular, is widely used in education due to its simple syntax and wide range of applications, making it an ideal language for beginners.In conclusion, the FOSS ecosystem is a dynamic and collaborative environment that supports innovation, learning, and development.

PART D - OPEN SOURCE VS PROPRIETARY

DIMENSION	PYTHON (OPEN SOURCE)	PROPRIETARY SOFTWARE
Cost	Completely free to use, no licensing fees	Requires paid licenses or subscriptions
Security	Source code is open and can be audited by anyone	Closed source, users must trust the company
Support	Community support, forums, documentation	Official paid support from company
Freedom To Modify	Full freedom to modify and customize	No permission to modify source code
Community vs Corporate Control	Community-driven development	Controlled by a single company
Flexibility	Highly flexible and adaptable	Limited flexibility

Python is completely free and open-source, which means there are no subscription fees, licensing costs, or usage restrictions. Anyone can download, use, and distribute **Python** without any financial burden. In contrast, proprietary software often comes with paid licenses or subscription models, which can become expensive over time, especially for students and small organizations. This makes Python a more accessible and cost-effective choice.

Another major advantage of Python is its transparency. Since the source code is publicly available, developers can examine it, understand how it works, and even improve it. This openness increases security because vulnerabilities can be identified and fixed quickly by the global community. On the other hand, proprietary software is closed-source, meaning users cannot inspect the code and must rely entirely on the company for updates and security fixes.

In terms of support, Python benefits from a vast and active community. Developers can find solutions through forums, documentation, and platforms like GitHub and Stack Overflow. This community-driven support system is highly effective and continuously growing. Proprietary software, however, provides official support services, but these are often paid and limited to the company's policies.

PROJECT TIMELINE

The project was completed in a structured manner over several weeks. Initially, the basic concepts of Linux and FOSS were studied to build a strong foundation. This was followed by learning Python and shell scripting techniques. In the next phase, scripts were developed and tested to ensure correct functionality. Finally, the report was prepared by organizing all theoretical and practical aspects in a systematic manner.

REFERENCES

1.Torvalds, L. (1991). Linux Kernel Development.

https://www.kernel.org

2.Van Rossum, G. Python Programming Language.
https://www.python.org
3.Free Software Foundation. The Free Software Definition.
https://www.gnu.org/philosophy/free-sw.html
4.Open Source Initiative. The Open Source Definition.
https://opensource.org/osd

5.Python Software Foundation. Python License Documentation.
https://docs.python.org/3/license.html
6.GitHub. About Open Source and Collaboration.
https://docs.github.com
7.The Linux Documentation Project. Linux System Documentation.
https://tldp.org
8.GeeksforGeeks. Linux and Python Tutorials.
https://www.geeksforgeeks.org

CONCLUSION

In conclusion, this project provided a comprehensive understanding of Linux and Free and Open Source Software (FOSS), along with their significance in modern computing. It highlighted how open-source systems promote transparency, collaboration, and innovation by allowing users to access and modify source code. The study of the **Linux** and the **Python** programming language demonstrated how open-source technologies have transformed software development and system operations. Through the analysis of licensing, ethical principles, and real-world applications, it became clear that FOSS plays a vital role in reducing costs and increasing accessibility. The practical implementation using shell scripting further enhanced the understanding of automation and system management in a Linux environment. Overall, this project not only strengthened theoretical knowledge but also improved practical skills, making open-source tools like Python and Linux essential for future learning and development.

SHELL SCRIPTS

Script 1 — System Identity Report

Purpose

The purpose of this script is to generate a detailed system identity report that displays important information about the user and the operating system. It collects data such as the system kernel version, current user, system uptime, date and time, and operating system details. This script helps in understanding the system environment and provides a quick overview of system configuration, which is useful for auditing and documentation purposes.

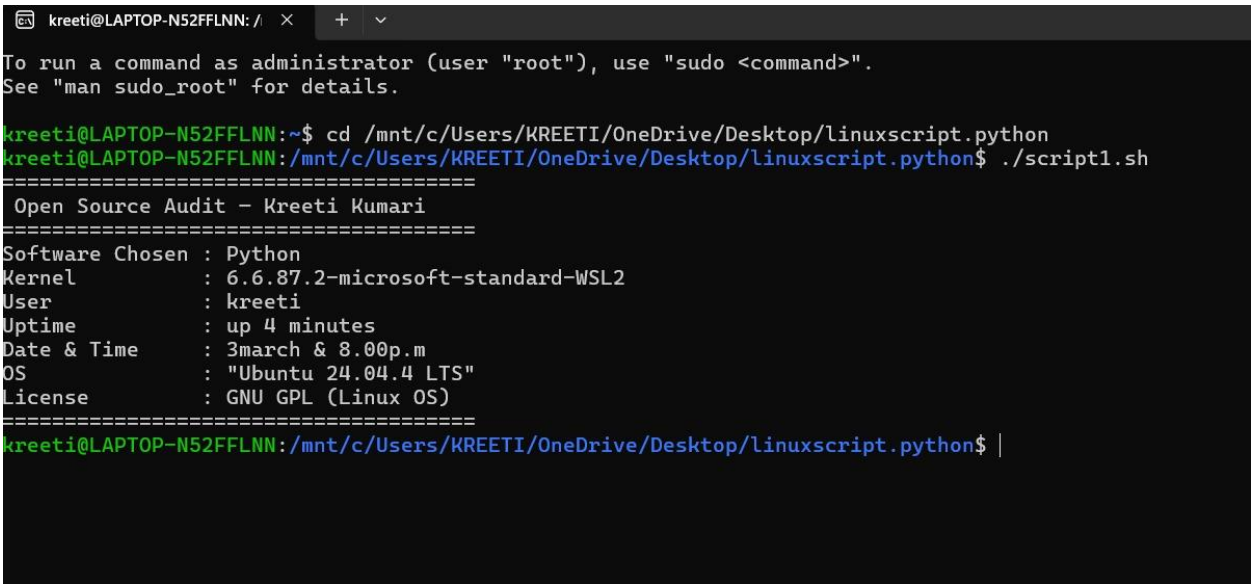
Concepts Used

This script uses basic shell scripting concepts such as **variables, command substitution, and system commands**. Commands like `uname`, `whoami`, `uptime`, and `date` are used to retrieve system information, while `/etc/os-release` is accessed to get OS details. The script also demonstrates the use of `echo` for formatted output. Overall, it shows how shell scripting can be used to collect and display system-level information efficiently.

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Your Name


STUDENT_NAME="Kreeti Thakur"

SOFTWARE_CHOICE="Python"


KERNEL=$(uname -r)

USER_NAME=$(whoami)

UPTIME=$(uptime -p)

DATE=$(date)

DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2)
```

```
echo "========================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================="
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Kernel          : $KERNEL"
echo "User            : $USER_NAME"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE"
echo "OS              : $DISTRO"
echo "License         : GNU GPL (Linux OS)"

echo "========================================="
```

screenshot:



Script 2 — Python Package Inspector

Purpose

The purpose of this script is to check whether a specific software package, particularly Python, is installed on the system. It verifies the availability of the package and displays its version if installed. Additionally, it provides a brief description of the package, helping users understand its purpose. This script is useful for ensuring that required tools are available before starting development work.

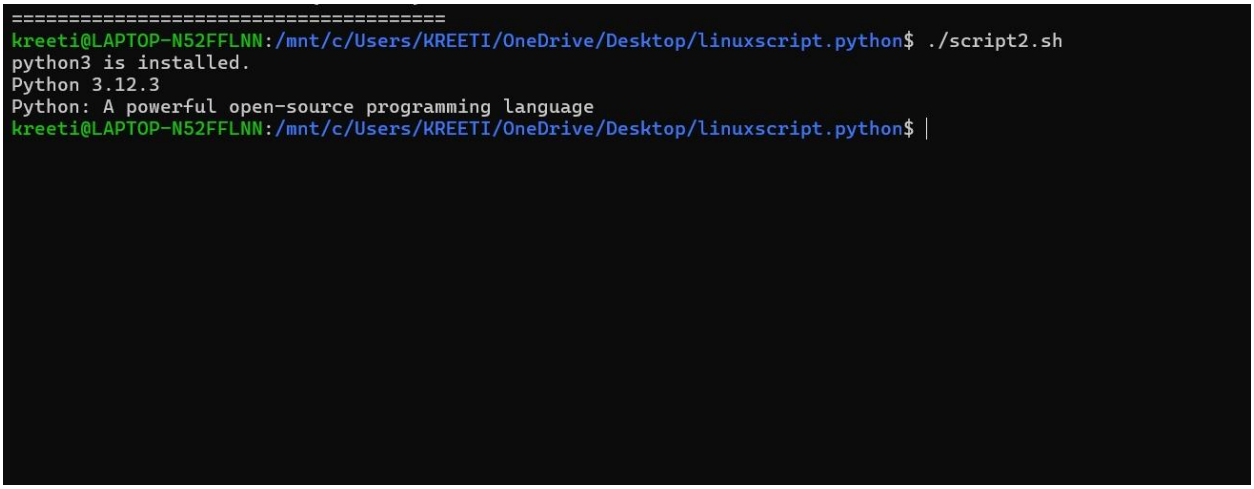
Concepts Used

This script uses **conditional statements and command checking**. The `command -v` command is used to check whether the package exists in the system. The `if-else` statement is used to handle different outcomes based on whether the package is installed or not. It also uses a `case` statement to display descriptions for different software packages. This demonstrates decision-making and branching in shell scripting.

```
#!/bin/bash

# Script 2: Python Package Inspector


PACKAGE="python3"


if command -v $PACKAGE &>/dev/null; then

  echo "$PACKAGE is installed."

  $PACKAGE --version

else

  echo "$PACKAGE is NOT installed."

fi
```

```
case $PACKAGE in
   python3) echo "Python: A powerful open-source programming language" ;;
   git) echo "Git: Version control system for developers" ;;
   firefox) echo "Firefox: Open-source web browser" ;;
   vlc) echo "VLC: Media player that plays everything" ;;
   *) echo "Unknown package" ;;
Esac
```

screenshot:



Script 3 — Disk and Permission Auditor

Purpose

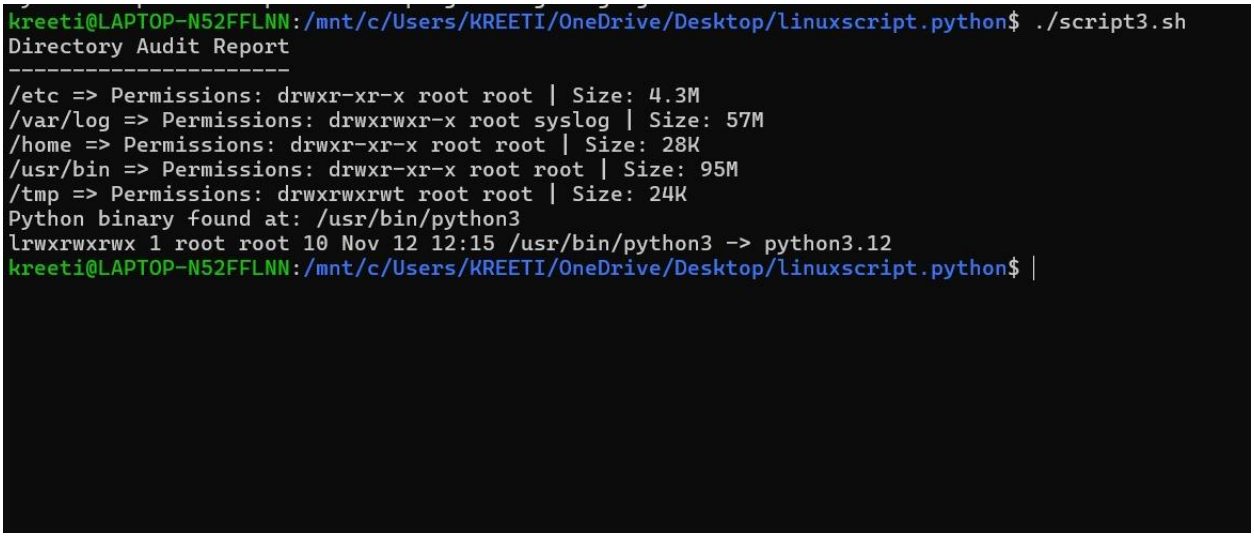
The purpose of this script is to analyze system directories by checking their permissions and disk usage. It provides a detailed report of important directories such as `/etc`, `/var/log`, `/home`, and others. Additionally, it verifies the presence of the Python binary and displays its file details. This script is useful for system monitoring, security auditing, and understanding resource usage.

Concepts Used

This script demonstrates **loops, arrays, and file handling concepts**. It uses an array to store directory paths and a `for` loop to iterate through them. Commands like `ls -ld` are used to check permissions, and `du -sh` is used to calculate directory size. The script also uses conditional statements (`if`) to verify directory existence and file presence. This shows how shell scripts can automate repetitive system checks efficiently.

```
#!/bin/bash

# Script 3: Disk and Permission Auditor


DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")


echo "Directory Audit Report"

echo "----------------------"


for DIR in "${DIRS[@]}"; do

  if [ -d "$DIR" ]; then

    PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

    echo "$DIR => Permissions: $PERMS | Size: $SIZE"

  else

    echo "$DIR does not exist"

  fi

done


# Python directory check
```

```
PYTHON_PATH=$(which python3)


if [ -f "$PYTHON_PATH" ]; then
   echo "Python binary found at: $PYTHON_PATH"
   ls -l $PYTHON_PATH
else
   echo "Python not found"
fi
```



Script 4 — Log File Analyzer

Purpose

The purpose of this script is to analyze log files and count the occurrences of a specific keyword, such as “error”. It reads the log file line by line and identifies matching entries. It also displays the last few matching lines, which helps in debugging and monitoring system issues. This script is particularly useful for system administrators to quickly identify errors in large log files.

Concepts Used

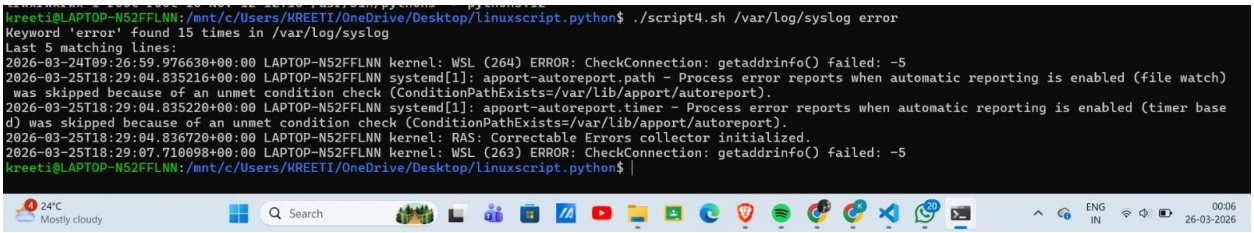
This script uses **file handling, loops, and string processing techniques**. It accepts input arguments for the log file and keyword, making it flexible. A `while` loop is used to read the file line by line, and `grep` is used for pattern matching. Arithmetic operations are used to count occurrences. It also demonstrates the use of default values and input validation. This script highlights how shell scripting can be used for text processing and analysis.

```
#!/bin/bash

# Script 4: Log File Analyzer


LOGFILE=$1

KEYWORD=${2:-"error"}

COUNT=0


if [ ! -f "$LOGFILE" ]; then

  echo "Error: File not found."

  exit 1

fi


while IFS= read -r LINE; do

  if echo "$LINE" | grep -iq "$KEYWORD"; then

    COUNT=$((COUNT + 1))

  fi

done < "$LOGFILE"


echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
```

echo "Last 5 matching lines:"

grep -i "$KEYWORD" "$LOGFILE" | tail –5

screenshot:



Script 5 — Open Source Manifesto Generator

Purpose

The purpose of this script is to generate a personalized open-source manifesto based on user input. It collects responses from the user about their daily tools, understanding of freedom, and future goals, and then creates a text file containing a structured message. This script promotes awareness of open-source philosophy and allows users to express their ideas creatively.

Concepts Used

This script uses **user input, file handling, and string formatting**. The `read` command is used to take input from the user, and variables are used to store responses. Output redirection (`>`, `>>`) is used to create and write content into a file. The script also uses the `date` command to include the current date in the output. This demonstrates how shell scripting can be used to interact with users and generate dynamic content.

#!/bin/bash

 # Script 5: Open Source Manifesto Generator

echo "Answer the questions:"

```
read -p "1. Tool you use daily: " TOOL

read -p "2. Meaning of freedom: " FREEDOM

read -p "3. What will you build: " BUILD



DATE=$(date '+%d %B %Y')

OUTPUT="manifesto_$(whoami).txt"



echo "On $DATE, I believe in open source." > $OUTPUT

echo "I use $TOOL daily." >> $OUTPUT

echo "For me, freedom means $FREEDOM." >> $OUTPUT

echo "I want to build $BUILD and share it with the world." >> $OUTPUT



echo "Manifesto saved to $OUTPUT"

cat $OUTPUT
```

screenshot:

```
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$ ./script5.sh
Answer the questions:
1. Tool you use daily: Python
2. Meaning of freedom: the ability to learn, modify, and share code without restrictions
3. What will you build: open-source projects that help students learn programming easily
Manifesto saved to manifesto_kreeti.txt
On 25 March 2026, I believe in open source.
I use Python daily.
For me, freedom means the ability to learn, modify, and share code without restrictions.
I want to build open-source projects that help students learn programming easily and share it with the world.
kreeti@LAPTOP-N52FFLNN:/mnt/c/Users/KREETI/OneDrive/Desktop/linuxscript.python$
```